

ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO BÀI TẬP LỚN

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

TheAllNewBinance

Môn học: Lập trình nâng cao (Java)

Học phần: 2526II_UET.CS2043_2

Nhóm: Nhóm 12

Thành viên nhóm:

Linh | Khánh | Minh | Đăng

Hà Nội, tháng 5 năm 2026

1. Giới Thiệu Mục Tiêu và Phạm Vi Thực Hiện

1.1 Mục tiêu

Dự án **TheAllNewBinance** được xây dựng nhằm mục tiêu phát triển một hệ thống đấu giá trực tuyến phân tán, hoàn chỉnh theo kiến trúc Client-Server, áp dụng các nguyên lý lập trình hướng đối tượng nâng cao và các tính năng hiện đại của **Java 21**.

Các mục tiêu cụ thể bao gồm:

- Xây dựng hệ thống đấu giá thời gian thực với cơ chế cập nhật giá thầu tức thì cho tất cả người dùng.
- Áp dụng các Design Pattern chuẩn (Factory Method, Observer/Pub-Sub) vào bài toán thực tế.
- Xử lý đồng thời (Concurrency) an toàn với Virtual Threads của Java 21, tránh Carrier Thread Pinning.
- Đảm bảo chất lượng mã nguồn thông qua Checkstyle, Spotless và CI/CD tự động.
- Triển khai các tính năng nâng cao: Anti-sniping, Bid History Chart thời gian thực.

1.2 Phạm vi thực hiện

Hệ thống bao gồm các chức năng chính:

- **Quản lý người dùng:** Đăng ký, đăng nhập, phân quyền (StandardUser / Admin).
- **Quản lý sản phẩm:** Tạo và phân loại sản phẩm đấu giá theo 3 nhóm (LuxuryCollectible, ArtisticCreation, PrecisionMechanical).
- **Phiên đấu giá:** Tạo phiên, đặt giá thầu, theo dõi trạng thái, kết thúc phiên.
- **Cập nhật thời gian thực:** Broadcast giá thầu mới đến tất cả client đang xem cùng phòng.
- **Quản lý ví:** Khóa số dư khi đặt thầu, hoàn tiền khi thua thầu.
- **Giao diện người dùng:** Ứng dụng desktop JavaFX với hỗ trợ dark/light theme.

2. Kiến Trúc Tổng Thể Hệ Thống

2.1 Kiến trúc Maven Multi-Module

Dự án được tổ chức theo kiến trúc **Maven Multi-Module**, tách biệt rõ ràng thành 3 module độc lập:

Module Core

Chứa các Domain Model, DTO và tiện ích dùng chung cho cả Client và Server.

- **Entity:** User (sealed: StandardUser, Admin), Item (sealed: LuxuryCollectible, ArtisticCreation, PrecisionMechanical), Auction, Bid.
- **Service Interfaces:** IUserService, IAuctionService, IBidService, IItemService.
- **DTO:** Các đối tượng truyền dữ liệu qua JSON (Gson).
- **Utils:** JsonMapper, DateFormatter, PasswordHasher (jBCrypt).

Module Server

Xử lý toàn bộ logic nghiệp vụ phía máy chủ, giao tiếp qua Socket TCP.

- **DAO Layer:** Interface + Implementation cho User, Item, Auction, Bid (MySQL).
- **Service Layer:** UserService, AuctionService, BidService, ItemService.
- **Controller Layer:** UserController, AuctionController, BidController, RequestDispatcher.
- **Network:** SocketServer xử lý kết nối đa luồng với Virtual Threads.
- **BroadcastBroker:** Pub/Sub phân phòng theo auctionId cho realtime updates.
- **DBExecutor:** Platform Thread Pool cô lập JDBC I/O tránh Carrier Pinning.
- **BidQueueManager:** Hàng đợi xử lý thầu riêng biệt cho từng phiên đấu giá.

Module Client

Giao diện người dùng desktop xây dựng bằng JavaFX theo mô hình MVVM.

- **Pages:** Login, Register, General (danh sách đấu giá), ProductDetail, Profile, Admin.
- **Components:** AuctionCard, Header, Navbar, SearchBar, ThemeToggle, BidHistoryChart.
- **Network:** AuctionClient (Socket), NetworkService, Correlation ID matching.

- **Services:** UserSessionService, AuctionQueryService, SceneService, NavigationService.

2.2 Luồng giao tiếp Client–Server

Client kết nối đến Server qua **Socket TCP**. Mỗi request được đóng gói dưới dạng JSON với *Correlation ID* để ghép cặp response. Server xử lý request qua chuỗi: **SocketServer** → **RequestDispatcher** → **Controller** → **Service** → **DAO** → **MySQL**.

Khi có sự kiện đấu giá (đặt thầu, gia hạn thời gian), **BroadcastBroker** tự động đẩy thông báo đến tất cả client đang subscribe cùng phòng đấu giá, đảm bảo cập nhật thời gian thực.

2.3 Sơ đồ kiến trúc tổng quan

Tầng	Module	Thành phần chính
Giao diện	client	JavaFX Pages, FXML, CSS, MVVM Controllers
Mạng Client	client	AuctionClient, NetworkService, Correlation ID
Mạng Server	server	SocketServer, RequestDispatcher, BroadcastBroker
Nghiệp vụ	server	UserService, AuctionService, BidService, ItemService
Dữ liệu	server	DAO Interfaces + Implementations, DBExecutor, MySQL
Domain chung	core	Entity, DTO, Service Interfaces, Utils

3. Các Chức Năng Đạt Được

a) Thiết kế lớp với Sealed Classes (Java 21)

Hệ thống phân cấp lớp được thiết kế sử dụng **Sealed Classes** của Java 21, đảm bảo an toàn kiểu tại compile-time. Lớp **User** được sealed với hai lớp con **StandardUser** và **Admin**, mỗi lớp override các phương thức nghiệp vụ như **canBid()**, **canSell()**, **canManageSystem()**. Tương tự, **Item** được sealed với 3 loại sản phẩm cụ thể.

Lý do lựa chọn: Sealed Classes cho phép compiler kiểm tra tính đầy đủ của switch expressions, loại bỏ các trường hợp không mong muốn và tăng tính bảo trì của code.

b) Design Pattern: Factory Method qua Java SPI

Thay vì dùng Factory Method truyền thống, nhóm áp dụng cơ chế **Java Service Provider Interface (SPI)** để đăng ký động các **Item Factory** (**ArtisticCreationFactory**, **LuxuryCollectibleFactory**, **PrecisionMechanicalFactory**). Thiết kế này tuân thủ triệt để nguyên lý **Open/Closed Principle**: thêm loại sản phẩm mới không cần sửa code hiện có.

c) Xử lý đồng thời: DBExecutor Pattern

Nhóm phát hiện vấn đề **Carrier Thread Pinning** khi dùng Virtual Threads với JDBC (MySQL Connector/J có các khối **synchronized** nội bộ). Giải pháp: tạo một **Platform Thread Pool cố định (DBExecutor)** khớp với kích thước connection pool để cô lập hoàn toàn blocking JDBC I/O, ngăn Virtual Thread bị ghim vào Carrier Thread.

Ngoài ra, **BidQueueManager** tạo hàng đợi xử lý thầu riêng biệt cho từng phiên đấu giá (**LinkedBlockingQueue**), tuần tự hóa các request đồng thời trước khi đẩy xuống DB, giảm thiểu Lock Contention và tăng throughput.

d) Hệ thống Exception phân tầng (Stackless Domain Exception)

Thiết kế hệ thống exception bài bản với base class **DomainException** (abstract, kế thừa **RuntimeException**), phân nhánh theo nghiệp vụ: **AuctionException**, **WalletException**, **UserException**. Override **fillInStackTrace()** trả về **this** (**Stackless Exception**) để tối ưu hiệu năng trong môi trường phân tán, tránh rò rỉ thông tin cấu trúc server.

e) Realtime: BroadcastBroker Pub/Sub

BroadcastBroker quản lý phòng đấu giá động theo **auctionId**. Client gửi **SUBSCRIBE_AUCTION** khi vào trang chi tiết và **UNSUBSCRIBE_AUCTION** khi rời trang. Khi có bid mới, broker broadcast tức thì đến tất cả client trong phòng. Tích hợp **Bid History LineChart** (JavaFX) cập nhật động theo thời gian thực.

f) Anti-sniping

Khi có bid mới trong khoảng thời gian nhạy cảm cuối phiên, hệ thống tự động gia hạn thêm 120 giây và broadcast thời gian kết thúc mới đến tất cả client qua BroadcastBroker, đảm bảo công bằng cho người tham gia.

4. Phân Chia Công Việc Giữa Các Thành Viên

Thành viên	Nhiệm vụ chính	Tỷ lệ đóng góp	Trạng thái
Khánh, Linh, Đăng	Backend Lead: Thiết kế và triển khai Server Socket, logic xử lý đa luồng, nghiệp vụ đấu giá, module core.	30%	Hoàn thành
Khánh	Frontend Lead: Xây dựng giao diện người dùng (UI/UX) bằng JavaFX, xử lý sự kiện, kết nối client-side tới server.	30%	Hoàn thành
Linh	Database & DAO: Thiết kế lược đồ CSDL (schema), triển khai DAO/DTO Pattern, viết các truy vấn SQL.	20%	Hoàn thành
Minh	Testing & Documentation: Viết Unit Test cho các chức năng quan trọng (JUnit), kiểm thử tích hợp, và soạn thảo tài liệu báo cáo.	20%	Hoàn thành

Công nghệ sử dụng: Java 21 | JavaFX | Maven Multi-Module | MySQL 8 | Gson 2.10.1 | jBCrypt 0.4 | JUnit 5 | GitHub Actions CI/CD | Spotless & Checkstyle